1.a.i

$$J(\theta) = \frac{1}{2}\beta\theta^2 \; ; \; \theta \in R, \alpha \text{ is +ve constant}$$

$$\theta^{[t]} = \theta^{[t-1]} - \alpha \nabla J(\theta^{[t-1]})$$

Let's first find derivative of $J(\theta)$ w.r.t θ

$$\nabla J(\theta) = \frac{d}{d\theta}\left[\frac{1}{2}\beta\theta^2\right] = \beta\theta$$

Clearly, $\nabla J(\theta) = \beta\theta$ is an unbounded in the domain of R
Hence it can't have a Lipschitz Constant.

Let plug in $\nabla J(\theta)$ in the iterative equation

$$\theta^{[t]} = \theta^{[t-1]} - \alpha\beta\theta^{[t-1]}$$

$$\theta^{[t]} = \theta^{[t-1]}[1 - \alpha\beta]$$

As we pass through multiple iterations of this equation $[t = 0, 1, 2 \cdots\cdots]$, we expect θ to be approaching to zero which is possible only if $|1 - \alpha\beta| < 1$

$$\text{ie } |1 - \alpha\beta| < 1$$

$$-1 < 1 - \alpha\beta < 1$$

which means

(1) $1 - \alpha\beta > -1 \Rightarrow 2 > \alpha\beta \Rightarrow \alpha < 2/\beta$

(2) $1 - \alpha\beta < 1 \Rightarrow -\alpha\beta < 0 \Rightarrow \alpha\beta > 0$

(2) which is possible only if α is greater than 0

therefore range of α is as follows:

$$\boxed{0 < \alpha < 2/\beta}$$

$$J(\theta) = \frac{1}{2} \beta \theta^2$$

$$\nabla J(\theta) = \frac{d}{d\theta} \left[\frac{1}{2} \beta \theta^2 \right] = \beta \theta.$$

$\frac{1}{2}\beta\theta^2$ is a convex function at minimum,

$$\nabla J(\theta) = 0$$

$$\therefore \beta\theta = 0 \Rightarrow \theta = 0.$$

i.e $J(\theta)$ converges at $\underline{\theta = 0}$

ie $\underline{\theta^{\dagger} = 0}$

1. a. ii

$$\theta^{[t]} = \theta^{[t-1]} - \alpha\beta\theta^{[t-1]}$$

T is the minimum number of iterations that we have to compute for convergence such that $|\theta^{[T]} - \theta^* \leq \varepsilon|$

$$\theta^{[T]} = \theta^{[0]}(1-\alpha\beta)^T$$

Applying Logarithm on both sides

$$\log\theta^{[T]} = \log\theta^{[0]} + T\log(1-\alpha\beta)$$

$$T = \frac{\log\theta^{[T]} - \log\theta^{[0]}}{\log(1-\alpha\beta)}$$

We know α' range is $0<\alpha<2/\beta$

1. with in $0<\alpha<1/\beta$; as α increases, $1-\alpha\beta$ gets more closer to zero. Hence the value of denominator $\log(1-\alpha\beta)$ is negative and decreases. Hence the value of T increases with α increasing within the range of $0<\alpha<1/\beta$.
2. Similarly, with in the range $1/\beta<\alpha<2/\beta$ the value of T decreases as α increases

simply put $(0, 1/\beta) \rightarrow \alpha$ increases, T increases
$(1/\beta, 2/\beta) \rightarrow \alpha$ increases, T decreases

1 · a · ii

It's very important to chose the appropriate α for the proper convergence at the minimum values as closely as possible such that $|\theta^{[T]} - \theta^*| \leq \epsilon$.

* Larger values of α makes the computation to completely ignore (or) step over the expected minimal value range (ie. $|\theta^{[T]} - \theta^*| \leq \epsilon$ thereby not serving the need of computing acceptable minimum value.

* smaller values of α makes it very time consuming and compute intensive to reach the convergence such that $|\theta^{[T]} - \theta^*| \leq \epsilon$.

Hence its important to chose the right value for α.

1.b

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{d} B_i \theta_i^2 \quad , \; B_i > 0 \text{ constant scalar}$$
$$B_i \in R$$

$$\nabla J(\theta) = \frac{1}{2} \cdot 2 B \theta = B\theta.$$

the gradiant descent rule

$$\theta_i^{[t]} = \theta_i^{[t-1]} - \alpha_i B_i \theta_i^{[t-1]}$$
$$\theta_i^{[t]} = (1 - \alpha B_i)\, \theta_i^{[t-1]}$$

for $\theta_i^{[t]}$ to converge at minimum,

$$|1 - \alpha B_i| < 1$$
$$-1 < 1 - \alpha B_i < 1$$

leading to $0 < \alpha < 2/B_i$

when GD converges, the gradiant should be zero

ie $\nabla J(\theta^+) = 0$

ie $B_i \theta_i^+ = 0$, sine all the $B_i > 0$,

the value of θ_i^+ at convergene is zero

$$\theta^+ = 0$$

I.C.ii

For both original trajectory as well as the rotated trajectory

1. The smaller values of the learningrate [more specifically learning rates of 0.05, 0.1, 0.2, 0.3, 0.4, 0.5] lead to succesful convergence whereas the larger values [0.5, 1.0] did not lead to convergence.

2. Among converged executions, the smaller the learning rate, the more the number of iterations required for convergence. For Example, for Rotated A, 0.05 learning rate required 531 iterations for convergence, 0.1 learning rate required 254 iterations for convergence and so on.

3. Among non - converged executions, we can not definitive
Say, if lesser the learning rate, lesser the no. of iterations
required to figure out convergence is not attainable.
In fact, Larger learningrate of 1.0 required 22
iterations to bail out where as learning rate of 0.5
took 3 iterations to bail out. This sort of indicates
divergence and overshooting associated with larger learning
rates.

$J(\theta)$ is convex, twice differentiable, Largest eigen value of its Hessian matrix $H = \nabla^2 J(\theta) \leq \beta_{max}$ for all points θ.

The Gradiant descent equation is

$$\theta^{[t]} = \theta^{[t-1]} - \alpha \nabla J(\theta^{[t-1]})$$

plugging in $x = \theta^{[t-1]}$, $y = \theta^{[t]}$, $y - x = -\alpha \nabla J(\theta^{[t-1]})$ in the taylor's polynomial from the hint, we get

$$J(\theta^{[t]}) = J(\theta^{[t-1]}) + \nabla J(\theta^{[t-1]})^T (-\alpha \nabla J(\theta^{[t-1]})) + \frac{1}{2}(-\alpha \nabla J(\theta^{[t-1]}))^T \nabla^2 J(\theta^{[t-1]} + c(\theta^{t} - \theta^{t-1})) \cdot (\theta^{[t]} - \theta^{[t-1]})$$

From taylor's theorem with Lagrange remainder,

$$J(\theta^{[t]}) = J(\theta^{[t-1]}) + \nabla J(\theta^{[t-1]})^T \cdot (-\alpha \nabla J(\theta^{[t-1]})) + \frac{1}{2}(-\alpha \nabla J(\theta^{[t-1]}))^T H \cdot (-\alpha \nabla J(\theta^{[t-1]}))$$

where $H = \nabla^2 J(\theta^{[t-1]} + c(\theta^{[t]} - \theta^{[t-1]}))$

$$J(\theta^{(t)}) = J(\theta^{[t-1]}) - \alpha (\nabla J(\theta^{[t-1]}))^2 + \frac{\alpha^2}{2} \nabla J(\theta^{[t-1]})^T \cdot H \nabla J(\theta^{[t-1]})$$

Hessian is a Symmetric matrix, the maximum value of its Quadratic form $(U^T H U)$ is the maximum value of its Eigen vector, where U is in its unit form.

$$\lambda_{max}(H) = \max(\underbrace{U^T H U}) \rightarrow \text{unit vector.}$$

$$\frac{\nabla J(\theta^{[t-1]})^T \cdot H \nabla J(\theta^{[t-1]})}{\| \nabla J(\theta^{[t-1]}) \|^2} \leq \beta_{max}.$$

1.d

$$\nabla J(\theta[t-1])^T H \nabla J(\theta[t-1]) \leq \beta_{max} \|\nabla J(\theta[t-1])\|^2$$

therefore plugging above in earlier equation, we get below inequality

$$J(\theta[t]) \leq J(\theta[t-1]) + \left(\frac{\alpha^2 \beta_{max}}{2} - \alpha\right) \|\nabla J \theta[t-1]\|^2$$

$$\boxed{J(\theta[t]) \leq J(\theta[t-1]) - \alpha\left(1 - \frac{\alpha \beta_{max}}{2}\right) \|\nabla J \theta[t-1]\|^2}$$

this inequation is in the form of iterative equation for gradient descent

For $J(\theta[t])$ to be strictly decreasing

$J(\theta[t])$ should be $< J(\theta[t-1])$

for any value of t.

Since Given $0 < \alpha < 1/\beta_{max}$

$$\alpha < \frac{1}{\beta_{max}}$$

$$\alpha \beta_{max} < 1 \Rightarrow -\frac{\alpha \beta_{max}}{2} > -\frac{1}{2}$$

$$\Rightarrow 1 - \frac{\alpha \beta_{max}}{2} > \frac{1}{2} \quad \text{ie } 1 - \frac{\alpha \beta_{max}}{2} > 0$$

positive value

ie to compute $J(\theta[t])$, we are subtracting a positive value from $J(\theta[t-1])$

ie $J(\theta[t]) = J(\theta[t-1])$ - positive value for all t's.

Hence, we can say $J(\theta)$ is a decreasing function

1.a Now coming to convergence proof,

for a strictly decreasing function, so long as we are taking a smaller values of α, we should definitely converge in an iterative implementation with trail and error.

2.a

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right]$$

where $y^{(i)} \in \{0,1\}$, $h_\theta(x) = g(\theta^T x)$, $g(z) = \frac{1}{1+e^{-z}}$

To find Hessian Matrix of $J(\theta)$, ie $\nabla^2 J(\theta)$, Lets Start with finding the first order partial derivative first.

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right]$$

Let $T_1(\theta) = \log(h_\theta(x^{(i)}))$

$$\frac{\partial T_1(\theta)}{\partial\theta_j} = \frac{\partial}{\partial\theta_j}\left(\log(h_\theta(x^{(i)}))\right) = \frac{\partial}{\partial\theta_j}\left(\log(g(\theta^T x))\right)$$

$$= \frac{1}{g(\theta^T x)} \cdot g'(\theta^T x) \cdot x_j$$

we know $g'(z) = g(z)\cdot(1-g(z))$, using this

$$\frac{\partial T_1(\theta)}{\partial\theta_j} = \frac{1}{\cancel{g(\theta^T x)}} \cdot \cancel{g(\theta^T x)}(1-g(\theta^T x)) \cdot x_j$$

$$\boxed{\frac{\partial T_1(\theta)}{\partial\theta_j} = \frac{\partial}{\partial\theta_j}\left(\log(h_\theta(x^{(i)}))\right) = 1-g(\theta^T x)\cdot x_j} \quad (1)$$

Let $T_2(\theta) = \log(1-h_\theta(x^{(i)}))$

$$\frac{\partial T_2(\theta)}{\partial\theta_j} = \frac{\partial}{\partial\theta_j}\left(\log(1-h_\theta(x^{(i)}))\right) = \frac{1}{1-g(\theta^T x)} \cdot -g'(\theta^T x)\cdot x_j$$

$$= \frac{\partial}{\partial\theta_j}\left(\log(1-g(\theta^T x))\right)$$

2.a

$$\frac{\partial T_2(\theta)}{\partial \theta_j} = \frac{-g'(\theta^T x)\, x_j}{1-g(\theta^T x)} = \frac{-g(\theta^T x)\cdot \cancel{(1-g(\theta^T x))}}{\cancel{(1-g(\theta^T x))}} \cdot x_j$$

$$\boxed{\therefore \frac{\partial T_2(\theta)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j}\left[\log\left(1-h_\theta(x^{(i)})\right)\right] = -g(\theta^T x)\cdot x_j} \quad \text{—(2)}$$

plugging (1) and (2) in $\frac{\partial J}{\partial \theta_j}$

$$\frac{\partial J}{\partial \theta_j} = \frac{-1}{n} \sum_{i=1}^{n} \left[y^{(i)}\left(1-g(\theta^T x)\right) x_j^{(i)} - (1-y^{(i)})\, g(\theta^T x)\, x_j \right]$$

$$= \frac{-1}{n} \sum_{i=1}^{n} \left[y^{(i)}\left[1-g(\theta^T x^{(i)})\right]\cdot x_j^{(i)} - (1-y^{(i)})\, g(\theta^T x^{(i)})\, x_j^{(i)} \right]$$

$$= \frac{-1}{n} \sum_{i=1}^{n} \left[y^{(i)} - \cancel{y^{(i)} g(\theta^T x^{(i)})} - g(\theta^T x^{(i)}) + \cancel{y^{(i)} g(\theta^T x^{(i)})} \right] x_j^{(i)}$$

$$= \frac{-1}{n} \sum_{i=1}^{n} \left[y^{(i)} - g(\theta^T x^{(i)}) \right]\cdot x_j^{(i)}$$

$$\therefore \frac{\partial J}{\partial \theta_j} = \frac{-1}{n} \sum_{i=1}^{n} \left[y^{(i)} - \underbrace{g(\theta^T x^{(i)})}_{h_\theta(x^{(i)})} \right]\cdot x_j^{(i)}$$

$$\boxed{\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^{n} \left[h_\theta(x^{(i)}) - y^{(i)} \right]\cdot x_j^{(i)}}$$

2.a) To find the Hessian Matrix of $J(\theta)$

ie $$\nabla^2 J(\theta) = \frac{\partial}{\partial \theta_k}\left[\frac{\partial J}{\partial \theta_j}\right] = \frac{\partial}{\partial \theta_k}\left[\frac{1}{n}\sum_{i=1}^{n}\left[h_\theta(x^{(i)}) - y^{(i)}\right].x_j^{(i)}\right]$$

$$= \frac{\partial}{\partial \theta_k}\left[\frac{1}{n}\sum_{i=1}^{n}\left[h_\theta(x^{(i)}) - y^{(i)}\right]\cdot x_j^{(i)}\right]$$

$$= \frac{\partial}{\partial \theta_k}\left[\frac{1}{n}\sum_{i=1}^{n}\left[g(\theta^T x^{(i)}) - \underbrace{y^{(i)}\Big]\cdot x_j^{(i)}}\right]$$

derivative becomes zero as no θ is involved.

$$\nabla^2 J(\theta) = \frac{1}{n}\sum_{i=1}^{n} g'(\theta^T x^{(i)})\cdot x_j^{(i)} x_k^{(i)}$$

$$\nabla^2 J(\theta) = \frac{1}{n}\sum_{i=1}^{n} g(\theta^T x^{(i)})\cdot\left(1 - g(\theta^T x^{(i)})\right)\cdot x_j^{(i)}\cdot x_k^{(i)}$$

$$\nabla^2 J(\theta) = \frac{1}{n}\sum_{i=1}^{n} h_\theta(x^{(i)})\cdot\left(1 - h_\theta(x^{(i)})\right)\cdot x_j^{(i)}\cdot x_k^{(i)}$$

$$\boxed{H = \nabla^2 J(\theta) = \frac{1}{n}\sum_{i=1}^{n} h_\theta(x^{(i)})\left(1 - h_\theta(x^{(i)})\right)\cdot x^{(i)} x^{(i)}}$$

is the Hessian of $J(\theta)$.

2.(a) To prove Hessian is positive semi definite.

ie $z^T H z \geq 0$. for any vector z

$$z^T H z = z^T \left[\frac{1}{n} \sum_{i=1}^{n} h_\theta(x^{(i)}) \left(1 - h_\theta(x^{(i)})\right) \cdot x^{(i)} x^{(i)} \right) \cdot z$$

$$= \frac{1}{n} \sum_{i=1}^{n} z^T \cdot \left[h_\theta(x^{(i)}) \cdot \left(1 - h_\theta(x^{(i)})\right) \right) \cdot x^{(i)} x^{(i)T}$$

$$= \frac{1}{n} \sum_{i=1}^{n} \left[h \cdot (1-h) \right] z^T \cdot x \cdot x^T \cdot z$$

$$= \frac{1}{n} \sum_{i=1}^{n} \left[h \cdot (1-h) \right] \cdot (z^T x)^2$$

$$z^T H z = \frac{1}{n} \sum_{i=1}^{n} \left[h \cdot (1-h) \right] \cdot \underbrace{(z^T x)^2}_{\text{is always positive as its squared ie } \geq 0}$$

is always positive

as its squared ie ≥ 0.

h is just a probability. Hence $h(1-h) \geq 0$.

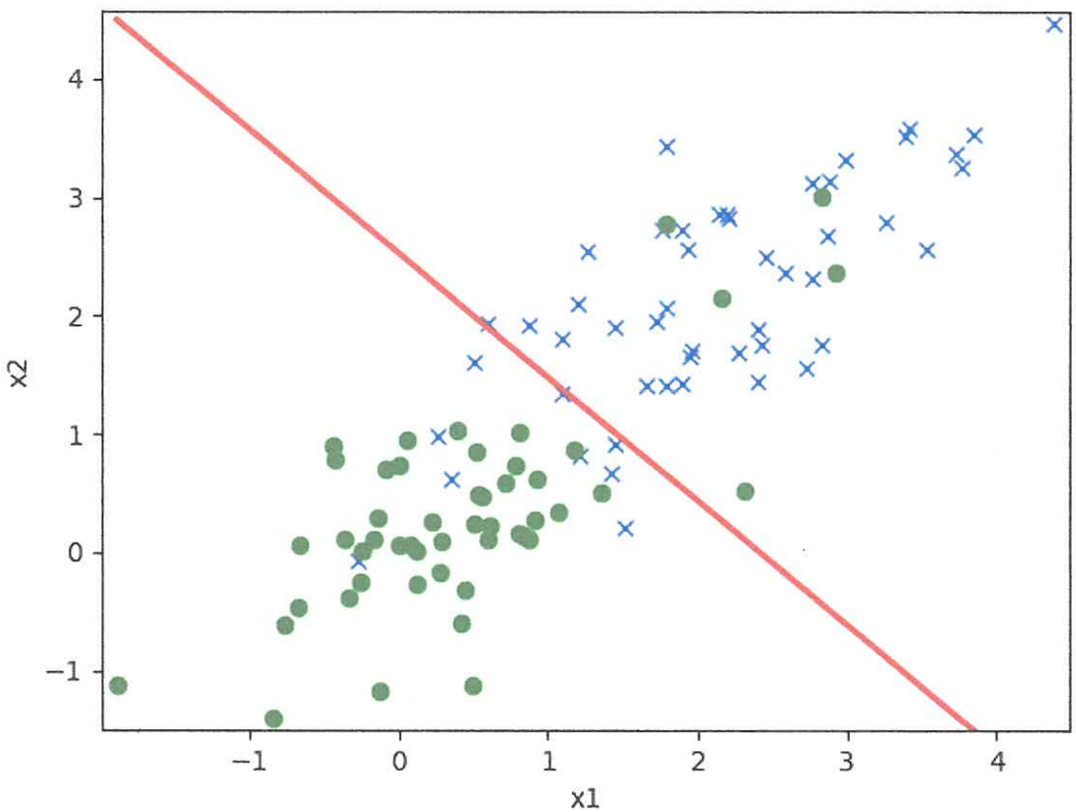
Since both terms are ≥ 0.

we can say $z^T H z \geq 0$ for any vector z

Hence the Hessian Matrix H of $J(\theta)$

is a positive semi definite.

2.b



2.b


```
iMac:linearclass dhruvakbhagi$ python3 logreg.py
```

// for first input ds1_train, ds1_valid
breaking the while loop as the theta_diff=[5.68308026e-11 1.36179649e-11 -2.30449104e-11]
is less than epsilon=1e-05
computed theta=[-2.40859977 1.03437317 0.24480705]
computed loss on training=0.40574136132209865

// for second input ds2_train, ds2_valid
breaking the while loop as the theta_diff=[3.92676373e-09 -1.70004008e-09 -1.44624615e-09]
is less than epsilon=1e-05
computed theta=[-2.09728529 0.86658135 0.82840081]
computed loss on training=0.4600081590961142

3.a

→ $P \in (0,1)$

→ Fraction P of examples in validation set are positive (with 1 as label)

→ Fraction $1-P$ of examples in validation set are negative (0 label)

$$\text{Accuracy} \triangleq \frac{\#\text{ Examples that are predicted correctly by classifier}}{\#\text{ Examples}}$$

i) For any dataset with p fraction of positive examples and $1-P$ fraction of negative examples, there exists a trivial classifier with accuracy $(1-P)$.

Imagine a trivial classifier C_T which predicts every input as negative (ie 0 label) irrespective of what the input is, in any given dataset.

Since by definition, any data set we are considering has $1-P$ fraction of negative examples/inputs, our trivial classifier C_T classifies all of those $(1-P)$ fraction inputs correctly as negative. Hence it will have the accuracy of $(1-P)$. Therefore any data set with P fraction of positive examples and $(1-P)$ negative examples, there exists a trivial classifier of accuracy $(1-P)$.

If 'n' is # of inputs in a dataset with P positive and $1-P$ negative as fractions,

$$\text{Accuracy} = \frac{n(1-P)}{n} = \underline{1-P}$$

3. a

$$A_1 \triangleq \frac{TP}{TP+FN} \qquad A_0 \triangleq \frac{TN}{TN+FP}$$

$$\text{Balanced Accuracy} = \bar{A} \triangleq \frac{1}{2}(A_0 + A_1) \quad -(1)$$

$$A = \frac{TP+TN}{TP+FN+TN+FP}$$

ii) Show $p = \dfrac{TP+FN}{TP+TN+FP+FN}$

* p is the fraction indicating the real positive samples irrespective of our classifier's prediction.
* out of True positive (TP), True negative (TN), False negative (FN), False positive (FP) and False negative (FN), the fraction denoting all positive samples/examples is

$$\boxed{\underbrace{TP}_{\text{positive classified as positive}} + \overbrace{FN}^{\text{positive classified as negative}}}$$

Let 'n' be total no. of samples, the fraction of positive samples in these 'n' is nothing but

$$p = \frac{\text{total no. of positive samples}}{\text{total no. of all the samples.}}$$

$$p = \frac{(TP+FN) \times \cancel{n}}{(TP+TN+FP+FN) \times \cancel{n}}$$

$$\boxed{\text{therefore } p = \frac{TP+FN}{TP+TN+FP+FN}}$$

3.a

ii) contd....

Show $A = p \cdot A_1 + (1-p) A_0$.

$$p \cdot A_1 + (1-p) A_0$$

$$= \frac{(TP+FN)}{(TP+TN+FP+FN)} \cdot \frac{TP}{(TP+FN)} + \left(1 - \frac{(TP+FN)}{TP+TN+FP+FN}\right) \times \frac{TN}{TN+FP}$$

$$= \frac{(TP+FN)}{(TP+TN+FP+FN)} \cdot \frac{TP}{(TP+FN)} + \left(\frac{\cancel{TP}+TN+FP+\cancel{FN}-\cancel{TP}-\cancel{FN}}{TP+TN+FP+FN}\right) \times \frac{TN}{TN+FP}$$

$$= \frac{(TP+FN)}{TP+TN+FP+FN} \cdot \frac{TP}{TP+FN} + \frac{\cancel{TN+FP}}{TP+TN+FP+FN} \cdot \frac{TN}{\cancel{TN+FP}}$$

$$= \frac{(TP+FN)}{TP+TN+FP+FN} \cdot \frac{TP}{TP+FN} + \frac{TN}{TP+TN+FP+FN}$$

$$= \frac{(TP)^2 + FN \cdot TP}{(TP+TN+FP+FN) \cdot (TP+FN)} + \frac{TN}{TP+TN+FP+FN}$$

$$= \frac{TP^2 + FN \cdot TP + TN \cdot TP + FN \cdot TN}{(TP+TN+FP+FN) \cdot (TP+FN)}$$

$$= \frac{TP(TP+FN) + TN(TP+PN)}{(TP+TN+FP+FN)(TP+FN)}$$

$$= \frac{(TP \cancel{+FN})(TP+TN)}{(TP+TN+FP+FN)(TP\cancel{+FN})}$$

$$= \frac{TP+TN}{TP+TN+FP+FN}$$

$$= A.$$

Therefore our starting point $p \cdot A_1 + (1-p)A_0 = A$

is ~~~~ has been proved.

3.a

iii Show trivial classifier in (i) has a balanced accuracy of 50%

Definition of balanced accuracy is given as $\bar{A} \triangleq \frac{1}{2}(A_0 + A_1)$.

Lets plug-in $A_0 = \frac{TN}{TN+FP}$, $A_1 = \frac{TP}{TP+FN}$

$$\bar{A} \triangleq \frac{1}{2}(A_0 + A_1)$$

$$\triangleq \frac{1}{2}\left[\frac{TN}{TN+FP} + \frac{TP}{TP+FN}\right]$$

our trivial classifier ~~only~~ classified every input as negative, it can't classify any input as positive.

Hence True positive $\rightarrow TP = 0$ and

False positive $\rightarrow FP = 0$.

plugging $TP = 0$, $FP = 0$ in above $\bar{A}$

$$\bar{A} \triangleq \frac{1}{2}\left[\frac{TN}{TN} + \frac{0}{FN}\right]$$

$$\bar{A} \triangleq \frac{1}{2}[1+0] = \frac{1}{2} = 50\%$$

Hence ~~our~~ balanced accuracy of our trivial classifier in (i) is 50%.

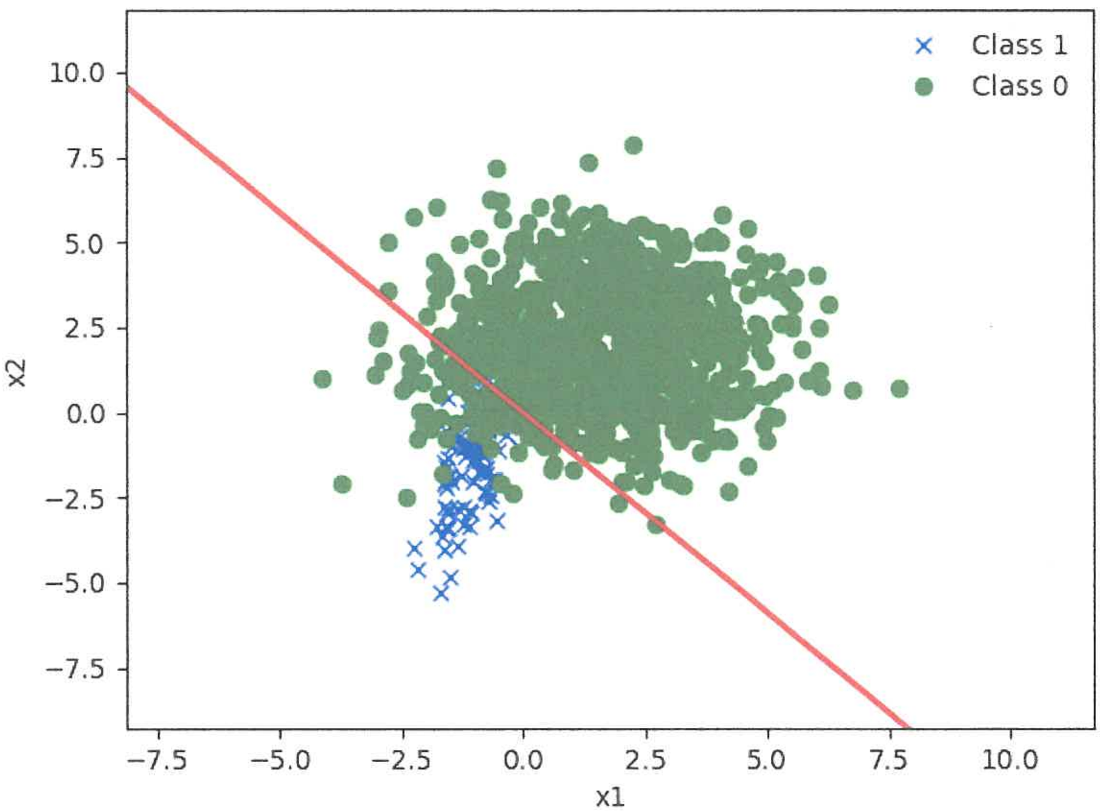
3.b

```
iMac:imbalanced dhruvakbhagi$ python3 imbalanced.py

accuracy=0.9472727272727273
balanced_accuracy=0.7775
accuracy_a_zero (negative)=0.985
accuracy_a_one (positive)=0.57
```

Observation :-

1. Minority class (positive class) accuracy is low. It is significantly lower than majority class Accuracy.
2. While accuracy is high, balanced accuracy is low. Balanced accuracy is a better measure



3.C Re-sampling / Re-weighting logistic regression.

Let n_p be # of positive examples in D.
Let n_n be # of negative examples in D.
Let n be total # of examples in D.

Fraction of positive examples in D is computed as

$$p = \frac{n_p}{n_p + n_n} = \frac{n_p}{n}.$$

Given $K = \frac{p}{1-p} \Rightarrow K = \frac{\frac{n_p}{n}}{\frac{n_n}{n}} = \frac{n_p}{n_n}$

$$\therefore \boxed{n_p = K n_n} \longrightarrow (1)$$

Balanced accuracy $= \frac{1}{2}(A_0 + A_1)$

where $A_1 \triangleq \frac{TP}{TP+FN}$, $A_2 \triangleq \frac{TN}{TN+FP}$

Balanced Accuracy of D $= \frac{1}{2}\left(\frac{TP}{n_p} + \frac{TN}{n_n}\right)$

Now focusing on D'

Let n'_n be no. of negative examples in D'.

Given $\boxed{n_n = n'_n} \longrightarrow (2)$

Let n'_p be no. of positive examples in D'.

Given $\boxed{n'_p = \frac{n_p}{K} = \frac{\cancel{K} \cdot n_n}{\cancel{K}} = n_n} \longrightarrow (3)$ (from (1))

Let n' be total number of samples in D'.

3.c

$$n' = n'_p + n'_n$$

from ② & ③, $n' = n_n + n_n = 2n_n$

Accuracy is defined as

$$A = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy of } D' = \frac{TP' + TN'}{n'} = \frac{TP' + TN'}{n'_n + n'_p}$$

$$\text{Accuracy of } D' = \frac{TP' + TN'}{2n_n} \quad (\text{from ② \& ③})$$

$$= \frac{1}{2}\left(\frac{TP' + TN'}{n_n}\right) = \frac{1}{2}\left[\frac{TP'}{n_n} + \frac{TN'}{n_n}\right]$$

Since given that for every positive example in D, there are $1/k$ repetitions in D' and the fact that our model and parameters haven't changed b/w D and D' then we can confidently say

$$TP' = \frac{TP}{k}.$$

With similar logic, we can confidently say

$TN' = TN$ (every negative sample appeared once and no change in model)

$$\therefore \text{Accuracy of } D' = \frac{1}{2}\left[\frac{TP}{k \cdot n_n} + \frac{TN}{n_n}\right]$$

from ① $k \cdot n_n = n_p$

$$\text{Accuracy of } D' = \frac{1}{2}\left[\frac{TP}{n_p} + \frac{TN}{n_n}\right]$$

3.c Accuracy of $D' = \frac{1}{2}\left[\frac{TP}{n_p} + \frac{TN}{n_n}\right]$

we already derived that the

Balanced Accuracy of $D = \frac{1}{2}\left[\frac{TP}{n_p} + \frac{TN}{n_n}\right]$

As the values we derived of Accuracy of D' and Balance Accuracy of D, we can confirm that

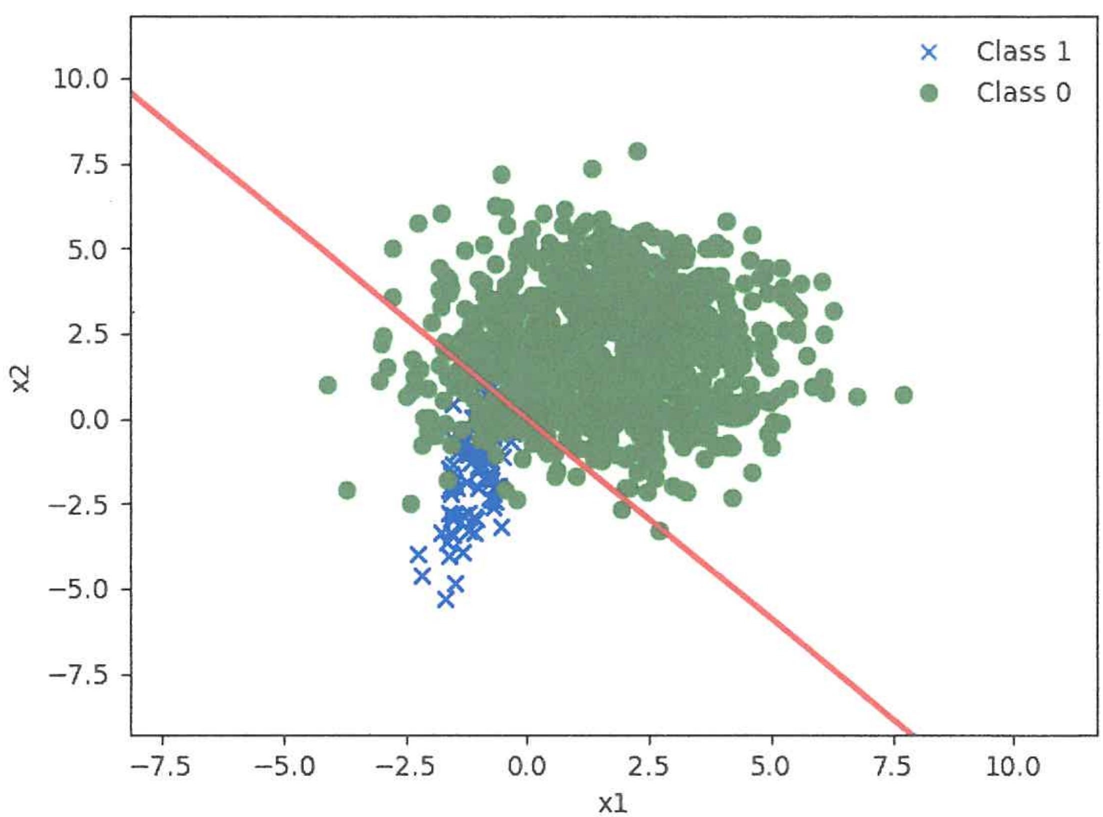
Balanced Accuracy of D = Accuracy of D'.

3.d

Observations

1. Accuracy on sampled validation set is equal to balanced accuracy on the original validation set.
2. Balanced accuracy on sampled validation set is same as that of original set.
3. A_0 (sampled validation set) is same as A_0 (original validation set)
4. A_1 (sampled validation set) is same as A_1 (original validation set).

accuracy_sampled=0.7775
balanced_accuracy_sampled=0.7775
accuracy_a_zero_sampled (negative)=0.985
accuracy_a_one_sampled (positive)=0.57



4.a

$$X = \begin{bmatrix} \text{---} x^{(1)} \text{---} \\ \text{---} x^{(2)} \text{---} \\ \vdots \\ \text{---} x^{(n)} \text{---} \end{bmatrix}_{n \times d} \quad \vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}_{n \times 1}$$

$$J_\lambda(\beta) = \frac{1}{2}\|X\beta - \vec{y}\|_2^2 + \frac{\lambda}{2}\|\beta\|^2$$

a) prove that $\beta_\lambda = (X^T X + \lambda I_{d\times d}) X^T \vec{y}$ (minimizer)

$$J_\lambda(\beta) = \frac{1}{2}\|X\beta - \vec{y}\|_2^2 + \frac{\lambda}{2}\|\beta\|^2$$

Can be written as

$$J_\lambda(\beta) = \frac{1}{2}(X\beta - \vec{y})^T (X\beta - \vec{y}) + \frac{\lambda}{2}\beta^T\beta.$$

$$= \frac{1}{2}\left[(\beta^T X^T - \vec{y}^T)(X\beta - \vec{y})\right] + \frac{\lambda}{2}\beta^T\beta$$

$$= \frac{1}{2}\left[\beta^T X^T X\beta - \underbrace{\beta^T X^T \vec{y} - \vec{y}^T X\beta}_{\text{scalars \& transposes are equal}} + \vec{y}^T \vec{y}\right] + \frac{\lambda}{2}\beta^T\beta$$

i.e. $\beta^T X^T \vec{y} = \vec{y}^T X\beta$

$$J_\lambda(\beta) = \frac{1}{2}\left(\beta^T X^T X\beta - 2\beta^T X^T \vec{y} + \vec{y}^T \vec{y}\right) + \frac{\lambda}{2}\beta^T\beta$$

$$J_\lambda(\beta) = \frac{1}{2}\beta^T X^T X\beta - \beta^T X^T \vec{y} + \frac{1}{2}\vec{y}^T \cdot \vec{y} + \frac{\lambda}{2}\beta^T\cdot\beta$$

Let's compute gradient of $J_\lambda(\beta)$ w.r.t β.

$$\nabla J_\lambda(\beta) = \frac{1}{\cancel{2}}(\cancel{2}X^T X\beta) - X^T\vec{y} + 0 + \frac{\lambda}{\cancel{2}}\cancel{2}\beta\cdot I$$

$$\nabla J_\lambda(\beta) = X^T X\beta - X^T\vec{y} + \lambda I_{d\times d}\beta$$

4.a

$$\nabla J_\lambda(\beta) = X^T X \beta - X^T \vec{y} + \lambda I_{d\times d}\beta$$

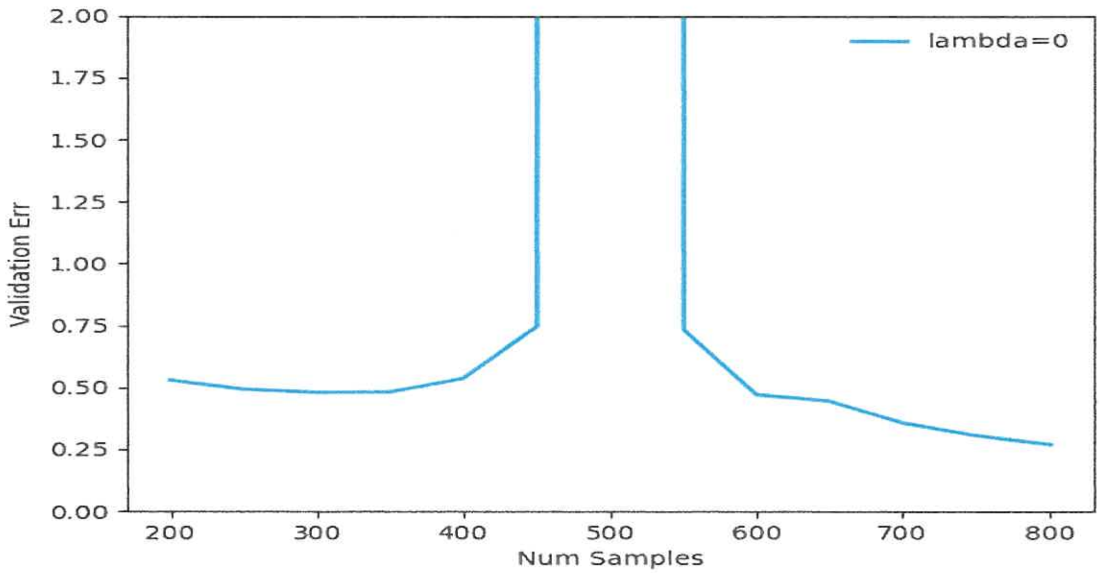
For closed form, we set $\nabla_\lambda J(\beta) = 0$ to find the minimum value (minimizer), Therefore.

$$X^T X \beta - X^T \vec{y} + \lambda I_{d\times d}\beta = 0.$$

$$(X^T X + \lambda I)\beta = X^T \vec{y}$$

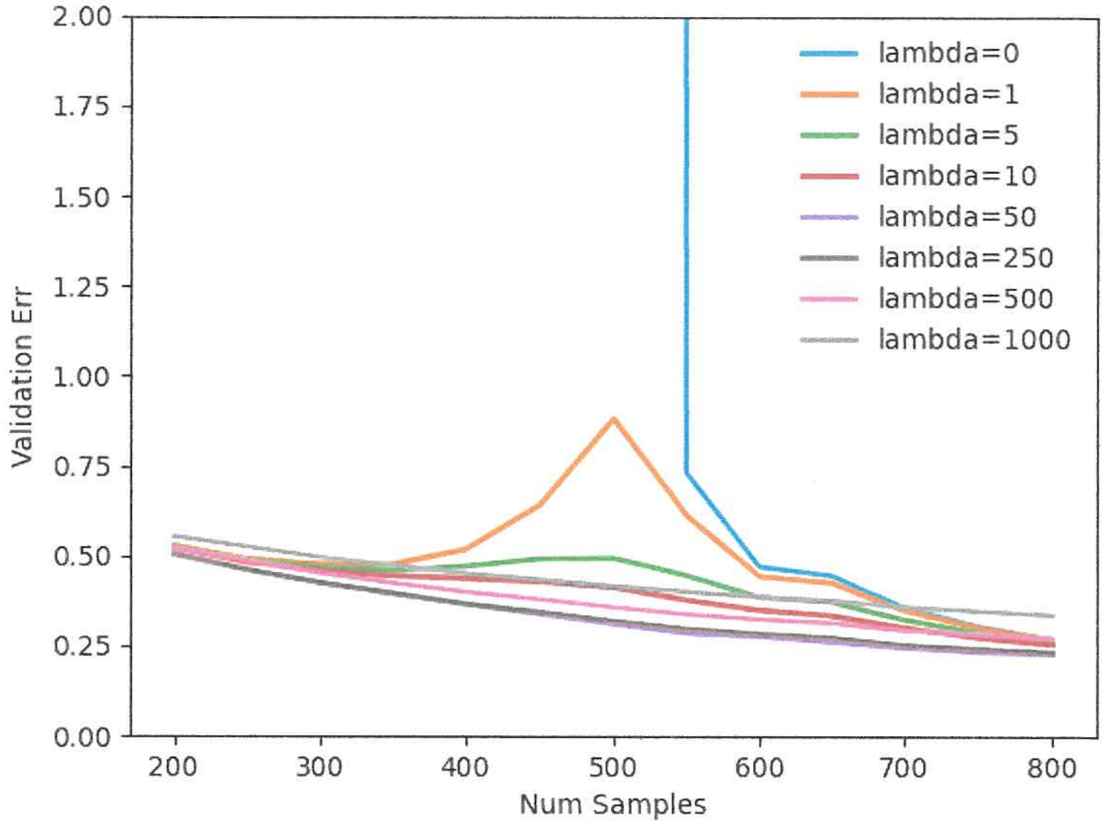
$$\boxed{\beta = (X^T X + \lambda I_{d\times d})^{-1} X^T \vec{y}}$$

4.b



Observation The validation error increases and then decreases as we increase the sample size.

4.c



observations

- For some smaller λ's ($\lambda=0$, $\lambda=1$, $\lambda=5$), the validation error increased and the decreased as we increase the sample size.

- Double descent does not occur for a larger values of λ (like $\lambda=1000$, $\lambda=500$, $\lambda=250$ ....)

- $\lambda=0$ shows a double descent case (for Ex.).